

```
In [ ]: import pandas as pd
import seaborn as sns
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt
```

These steps were followed:

1. State the Null and the Alternative Hypotheses.
2. Define your alpha.
3. Collect data that is random and independent
4. Calculate the test result
5. Interpret the test result

Hypothesis testing: Chi-Square Test within the Eniac case study In this notebook we perform a chi-square test with the data from the Eniac case study, applying a post-hoc correction to perform pairwise tests and find the true winner.

1. State the Null Hypothesis and the Alternative Hypothesis.

Null Hypothesis H0: All versions have the same CTR.

Alternative Hypothesis HA: There is a difference in the CTR for the different versions.

2. Select an appropriate significance level alpha (α).

It was decided that a relatively high alpha was acceptable in this case

```
In [ ]: alpha = 0.05
```

3. Collect data that is random and independent

The important pieces of information (clicks on each element of interest & visits on each page) are scattered around. Let's collect them. Where are the .csv files?

```
In [ ]: eniac_a = "C:\\Users\\pakan\\OneDrive\\DATA_ SCIENCE_ BOOTCAMP_WBS\\DATA SCIENCE Ma
sample_a = pd.read_csv(eniab_a)
eniab_b = "C:\\Users\\pakan\\OneDrive\\DATA_ SCIENCE_ BOOTCAMP_WBS\\DATA SCIENCE Ma
```

```
sample_b = pd.read_csv(eniab_b)
eniab_c = "C:\\Users\\pakan\\OneDrive\\DATA_ SCIENCE_ BOOTCAMP_WBS\\DATA SCIENCE M
sample_c = pd.read_csv(eniab_c)
eniab_d = "C:\\Users\\pakan\\OneDrive\\DATA_ SCIENCE_ BOOTCAMP_WBS\\DATA SCIENCE Ma
sample_d = pd.read_csv(eniab_d)
```

```
In [ ]: pd.set_option('display.max_colwidth', None)
```

```
In [ ]: #sample_a
```

Clicks are stored in row Shop NOW/SeeDeals (column Name), extract number of clicks from column No.Clicks

```
In [ ]: a_clicks = sample_a.loc[sample_a['Name'] == 'SHOP NOW', 'No. clicks'].values[0]
```

```
In [ ]: # using a for loop to get click of a 4 samples at once
clicks=[]
versions=['SHOP NOW','SEE DEALS']
for df in [sample_a, sample_b, sample_c, sample_d]:
    clicks_1 = df.loc[df['Name'] .isin(versions), 'No. clicks'].iloc[0]
    clicks.append(clicks_1)
clicks
```

```
In [ ]: #Number of visits
sample_a.loc[1, 'Snapshot information']
```

```
In [ ]: # automatically using regex, manually if completly fine, too!
visits = []
for df in (sample_a, sample_b, sample_c, sample_d):
    # Extracts the numeric count from a string containing one or more digits followed
    val = df.loc[df['Name'] == 'mySidebar', 'Snapshot information'].str.extract(r'(\d+)')
    visits.append(int(val.item()))

visits
```

```
In [ ]: #set up dataframe/contengency table
observed_results = pd.DataFrame(
    [clicks, visits], columns=["A", "B", "C", "D"], index=["Clicks", "Visits"]
)
observed_results.loc['No_clicks'] = observed_results.loc['Visits'] - observed_results
observed_results
```

```
In [ ]: data = observed_results.loc[['Clicks', 'No_clicks']]
data
```

4. Calculate the test result

```
In [ ]: chisq, pvalue, df, expected = stats.chi2_contingency(data)
```

```
In [ ]: chisq
```

```
In [ ]: pvalue
```

5. Interpret the test result

The p-value is smaller than alpha. We reject the null hypothesis. There is a sufficient evidence to suggest at least one of the web pages has a significantly different click through rate compared to the others. Identify the better performing website with post-hoc test.

How do we decide who's the winner?

```
In [ ]: import itertools

combos = list(itertools.combinations(data, 2))

for combo in combos:
    print(f"{combo[0]} vs {combo[1]}", end=': ')
    res = stats.chi2_contingency(data.loc[:,list(combo)])
    if res.pvalue < alpha / len(combos):
        print(f"Significant. p = {res.pvalue:.3f}")
    else:
        print(f"NOT significant. p = {res.pvalue:.3f}")
```

Calculate and plot CTRs

```
In [ ]: # CTR rate
observed_results.loc['CTR'] = observed_results.loc['Clicks'] / observed_results.loc['Visits']
observed_results.sort_values(ascending=False, by='CTR', inplace=True)
```

```
In [ ]: observed_results.loc['CTR'].sort_values(ascending=False).plot(kind='bar');
```

```
In [ ]: # 1. Calculate Standard Error and Margin of Error (95% CI)
# Formula: 1.96 * sqrt( (p * (1-p)) / n )
z = 1.96
observed_results_T = observed_results.T # Transpose so versions are rows
p = observed_results_T['CTR']
n = observed_results_T['Visits']

moe = z * np.sqrt((p * (1 - p)) / n)

# 2. Plotting
plt.figure(figsize=(10, 6))
order = p.sort_values(ascending=False).index

ax = sns.barplot(
    x=order,
    y=p.loc[order],
    hue=observed_results_T.index,
    palette=['skyblue' if webp == 'A' else 'lightgrey' for webp in order])
```

```
)  
  
# 3. Add the manual error bars  
plt.errorbar(  
    x=np.arange(len(order)),  
    y=p.loc[order],  
    yerr=moe.loc[order],  
    fmt='none', c='black', capsize=5  
)  
  
plt.title('Version A and C have the highest CTR and their 95% Confidence Interval o  
plt.ylabel('Click-Through Rate')  
plt.xlabel('Website Version')  
plt.show()
```